it from our code.[5] Here, the risk is minimal. The method we're modifying is part of RSpec's published formatter protocol, and therefore won't change without a major RSpec version bump. In general, though, monkey-patching should be a tool of last resort.

This small amount of Ruby code is all it takes to define and configure this formatter so that you can use it from the command line. Now, let's move on to the other major way to configure RSpec: the `configure` method.

RSpec.configure

You've seen how easily you can set configuration options for a particular spec run via the command line. You've also seen how to make your favorite options the default using `.rspec` files.

As convenient as they are, command-line flags are not available for all RSpec options—just the ones you're likely to change from run to run. For the rest, you'll need to call `RSpec.configure` inside one or more Ruby files. You can have multiple `configure` blocks in your code base; if you do, RSpec will combine the options from all of them.

On a typical project, you'll put setup in `spec/spec_helper.rb` and then load this file automatically by adding `--require spec_helper` to your `.rspec` file.

Be Careful What You Load from spec_helper.rb



It's easy for your `spec_helper` file to get bogged down with code that you don't need for every spec, turning a spec run that would normally finish in hundreds of milliseconds into a multisecond "I wonder what's interesting on Twitter?" slog.

You'll have a *much* more enjoyable TDD experience if you limit `spec_helper` to load just the dependencies you *always* want. If you need a library just for a subset of specs, load it conditionally by doing one of the following:

- Add a `when_first_matching_example_defined` hook inside your `Rspec.configure` block
- `require` your library from the top of spec files that need it

You've used `RSpec.configure` several times as you've worked through the examples in this book. We'll do a quick review of the techniques you've seen, and in the process we'll show you a few more options.

5. http://culttt.com/2015/06/17/what-is-monkey-patching-in-ruby/